HIGH LEVEL DESIGN DOCUMENT

Supply Chain Management Platform

Enterprise B2B Microservices Architecture — End-to-End Design

Version: 2.0 | **Date:** April 2026

Architecture: Microservices with Event-Driven Integration

Stack: .NET 10 · Angular 19 · SQL Server · RabbitMQ · Redis

Classification: Confidential — Internal Use Only

Table of Contents

1. Executive Summary & Business Context
2. Scope Definition
3. User Roles & Access Matrix
4. Architecture Principles
5. Technology Stack
6. System Context Diagram (C4 Level 1)
7. Container Diagram (C4 Level 2)
8. Domain Decomposition
9. Integration Architecture
10. Data Architecture
11. Security Architecture
12. Deployment Architecture
13. Non-Functional Requirements
14. Notification & Email Trigger Architecture
15. AI Augmentation — Exception Playbooks
16. Risks & Mitigations
17. Architectural Decisions Record (ADR)

1. Executive Summary & Business Context

The **Supply Chain Management Platform** is an enterprise-grade B2B SaaS solution that digitizes the complete dealer lifecycle — from onboarding and catalog browsing to order placement, logistics tracking, invoicing, and payment collection.

The system is built on a **microservices architecture** with 6 independently deployable domain services, an API gateway, and a modern Angular SPA. It leverages event-driven communication via RabbitMQ for cross-domain consistency, Redis for caching and idempotency, and SQL Server for transactional persistence.

> **Key Business Capabilities:** Dealer onboarding & credit management · Product catalog with inventory control · Order lifecycle with credit-hold decisioning · Shipment tracking with agent assignment · GST-compliant invoice generation · Multi-channel notification delivery

2. Scope Definition

2.1 In Scope

- Identity & access management with JWT authentication and RBAC
- Product catalog with category hierarchy and stock tracking
- Order lifecycle management with state machine transitions
- Logistics shipment orchestration with agent/vehicle assignment
- Payment gateway integration (Razorpay), credit management, invoice PDF generation
- Multi-channel notification engine (in-app + email)
- API gateway with rate limiting, correlation tracking, and load balancing
- Role-based Angular SPA with real-time dashboards

2.2 Out of Scope

- External ERP/SAP integration beyond current API contracts
- Multi-region active-active deployment
- Mobile native applications (iOS/Android)
- Advanced ML-based fraud detection models

3. User Roles & Access Matrix

ROLE	RESPONSIBILITIES	KEY ACCESS
Admin	Platform governance, dealer approval, catalog management, full operational visibility	All services — full CRUD
Dealer	Browse catalog, place orders, track shipments, view invoices, make payments	Own data — scoped read/write
Warehouse	Stock management, order fulfillment, inventory alerts	Catalog stock + Order queue
Logistics	Shipment assignment, status progression, delivery orchestration	All shipments + Order status
Agent	Last-mile delivery execution, shipment status updates	Assigned shipments only

4. Architecture Principles

PRINCIPLE	DESCRIPTION
Domain-Driven Decomposition	Each microservice owns a single bounded context with its own database
Database-per-Service	No shared schemas — services communicate via APIs and events only
Outbox Pattern	Reliable event publication using transactional outbox with background dispatchers
Event-Driven Integration	RabbitMQ topic exchange for asynchronous cross-domain communication
CQRS-Lite	MediatR pipeline with validation, idempotency, transaction, and logging behaviors
Zero-Trust Auth	JWT validation at gateway and every service, with internal API key protection
Idempotent Operations	Redis-backed idempotency keys prevent duplicate processing

5. Technology Stack

BACKEND RUNTIME

.NET 10 (C# 13)

ASP.NET Core Web APIs with minimal hosting model

FRONTEND FRAMEWORK

Angular 19

Standalone components, signals, lazy-loaded modules

API GATEWAY

Ocelot Gateway

Route mapping, JWT pass-through, rate limiting, round-robin LB

PRIMARY DATABASE

SQL Server 2022

Dedicated database per microservice · EF Core Code-First

MESSAGE BROKER

RabbitMQ 4.1

Topic exchange with outbox pattern · Dead-letter queues

CACHE / IDEMPOTENCY

Redis 7.4

Token revocation store, idempotency keys, response caching

APPLICATION PIPELINE

MediatR + FluentValidation

Cross-cutting behaviors: logging, validation, transactions, idempotency

OBSERVABILITY

Serilog + Health Checks

Structured logging, gateway metrics, per-service health endpoints

PAYMENT GATEWAY

Razorpay

Order creation, signature verification, optional capture

PDF ENGINE

QuestPDF

GST-compliant invoice PDF generation with branded templates

EMAIL TRANSPORT

MailKit (SMTP)

Branded HTML email templates with digest batching

CONTAINERIZATION

Docker Compose

Redis, RabbitMQ, Mailpit orchestrated via docker-compose.yml

6. System Context Diagram (C4 Level 1)

This diagram shows the platform's external boundaries, actors, and third-party integrations.

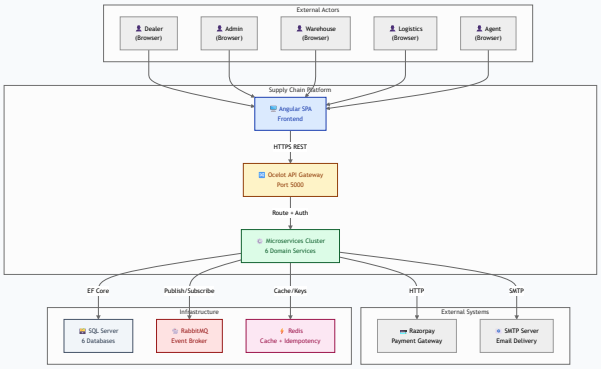

FIGURE 1 — SYSTEM CONTEXT DIAGRAM (C4 LEVEL 1)

7. Container Diagram (C4 Level 2)

Detailed view of all deployable containers, their ports, and communication patterns.

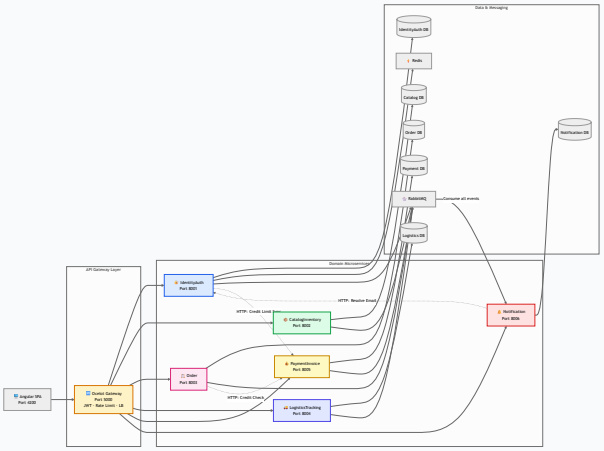

FIGURE 2 — CONTAINER DIAGRAM (C4 LEVEL 2) — ALL DEPLOYABLE UNITS

7.1 Service Port Allocation

SERVICE	PRIMARY PORT	LB INSTANCE PORT	DATABASE
Ocelot Gateway	5000	—	—
IdentityAuth	8001	8101	IdentityAuthDb
CatalogInventory	8002	8102	CatalogInventoryDb
Order	8003	8103	OrderDb
LogisticsTracking	8004	8104	LogisticsTrackingDb
PaymentInvoice	8005	8105	PaymentInvoiceDb
Notification	8006	8106	NotificationDb

8. Domain Decomposition

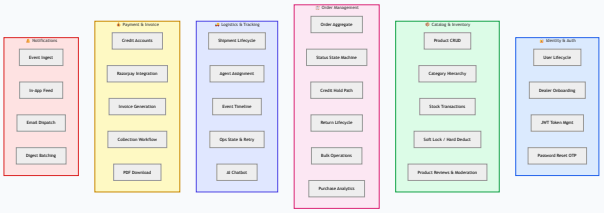

FIGURE 3 — DOMAIN DECOMPOSITION — BOUNDED CONTEXTS & CAPABILITIES

9. Integration Architecture

9.1 Synchronous Integrations (HTTP)

SOURCE	TARGET	PURPOSE	PROTOCOL
Order Service	PaymentInvoice	Credit check before order placement	HTTP GET
IdentityAuth	PaymentInvoice	Sync credit limit on dealer approval	HTTP PUT (Internal Key)
Notification	IdentityAuth	Resolve recipient email for delivery	HTTP GET (Internal Key)
Angular SPA	All Services	All user interactions via Gateway	HTTPS REST

9.2 Asynchronous Integrations (Event-Driven)

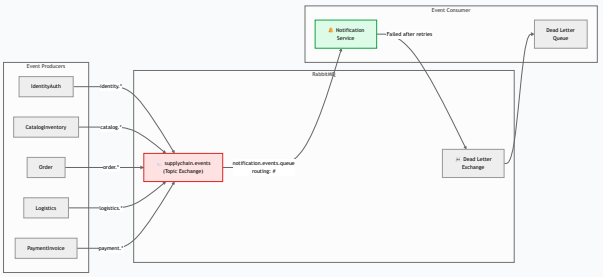

FIGURE 4 — EVENT-DRIVEN INTEGRATION VIA RABBITMQ TOPIC EXCHANGE

9.3 Event Routing Keys

SERVICE	ROUTING KEY PATTERN	EXAMPLE EVENTS
IdentityAuth	`identity.<eventType>`	DealerApproved, DealerRejected, PasswordReset
CatalogInventory	`catalog.<eventType>`	ProductCreated, StockUpdated, LowStockAlert
Order	`order.<eventType>`	OrderPlaced, OrderCancelled, ReturnRequested
LogisticsTracking	`logistics.<eventType>`	ShipmentCreated, StatusUpdated, Delivered
PaymentInvoice	`payment.<eventType>`	InvoiceGenerated, PaymentVerified

10. Data Architecture

10.1 Database Ownership Model

> **Principle:** Each microservice owns its dedicated SQL Server database. No shared schemas. Cross-service data access occurs exclusively through published APIs or domain events.

DATABASE	OWNER SERVICE	CORE TABLES
IdentityAuthDb	IdentityAuth	Users, DealerProfiles, RefreshTokens, OtpRecords, OutboxMessages
CatalogInventoryDb	CatalogInventory	Products, Categories, StockTransactions, StockSubscriptions, OutboxMessages
OrderDb	Order	Orders, OrderLines, OrderStatusHistory, ReturnRequests, OutboxMessages
LogisticsTrackingDb	LogisticsTracking	Shipments, ShipmentEvents, ShipmentOpsStates, OutboxMessages
PaymentInvoiceDb	PaymentInvoice	DealerCreditAccounts, Invoices, InvoiceLines, WorkflowStates, PaymentRecords, OutboxMessages
NotificationDb	Notification	Notifications, OutboxMessages

10.2 Consistency Strategy

SCOPE	MODEL	MECHANISM
Within Service	**Strong Consistency**	SQL Server transactions via EF Core SaveChangesAsync
Across Services	**Eventual Consistency**	Outbox pattern → RabbitMQ → Consumer processing

11. Security Architecture

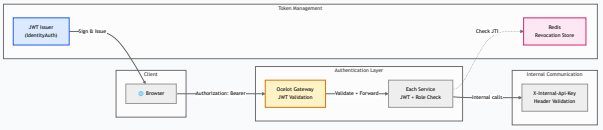

FIGURE 5 — SECURITY ARCHITECTURE — AUTHENTICATION & AUTHORIZATION FLOW

LAYER	MECHANISM	DETAILS
Authentication	JWT Bearer	Issuer, audience, signature, and lifetime validation
Authorization	RBAC	Controller-level [Authorize(Roles)] + frontend route/action guards
Token Revocation	Redis store	Identity service checks JTI against revocation set
Internal API	API Key header	X-Internal-Api-Key for service-to-service calls
Password Storage	Hashing only	No cleartext persistence — bcrypt-style hashing
Rate Limiting	Ocelot	Per-route and global rate limits, keyed by Oc-Client header

12. Deployment Architecture

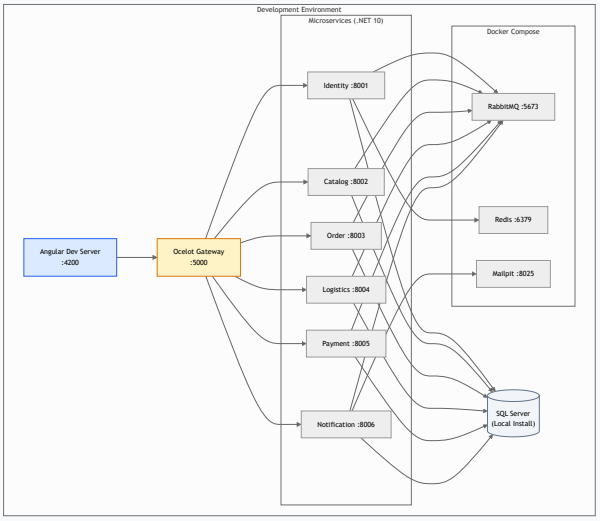

FIGURE 6 — LOCAL DEVELOPMENT DEPLOYMENT TOPOLOGY

13. Non-Functional Requirements

⏱️ Performance

p95 read latency < 300ms · p95 write latency < 500ms for standard commands

📈 Scalability

Stateless API nodes scale horizontally · RabbitMQ + Redis for shared state · Round-robin LB supported

🛡️ Availability

Target 99.9% for core API workflows in single-region setup

🔒 Security

RBAC for all endpoints · JWT + Redis revocation · Audit logging for admin actions

🔧 Maintainability

Clean architecture per service · Shared MediatR behaviors · EditorConfig for code style

🧪 Testability

xUnit domain tests per service · JUnit XML output · Repository abstraction for mocking

14. Notification & Email Trigger Architecture

The notification engine processes domain events from all services via RabbitMQ. Events are classified into **In-App** (default) or **Email** channels based on a routing policy.

14.1 Email Trigger Matrix

SOURCE SERVICE	EVENT	TRIGGER CONDITION	RECIPIENT
Identity	DealerRegistered	Dealer self-registration submitted	Payload email
Identity	PasswordResetRequested	OTP reset flow initiated	Payload email
Identity	PasswordResetCompleted	Password reset success	Payload email
Identity	DealerApproved	Admin approves dealer	Payload email
Identity	DealerRejected	Admin rejects dealer	Payload email
Order	OrderPlaced	Order accepted after credit check	Identity lookup
Order	AdminApprovalRequired	Order exceeds credit limit	Identity lookup
Order	OrderCancelled	Dealer/admin cancellation	Identity lookup
Order	ReturnRequested	Dealer requests return	Identity lookup
Logistics	ShipmentCreated	Shipment created for order	Identity lookup
Logistics	ShipmentAssigned	Agent assigned to shipment	Identity lookup
Logistics	ShipmentStatusUpdated	Any shipment status change	Identity lookup
Payment	InvoiceGenerated	Invoice generated for order	Identity lookup
Payment	CreditLimitUpdated	Dealer credit limit changed	Identity lookup

14.2 Email Processing Pipeline

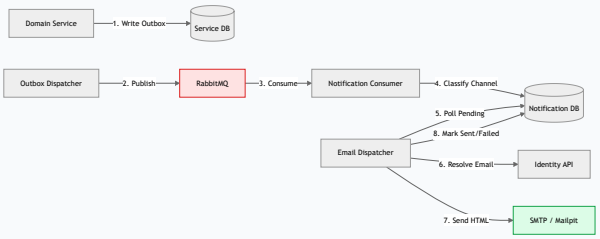

FIGURE 7 — EMAIL NOTIFICATION PROCESSING PIPELINE (8-STEP FLOW)

> **In-App vs Email:** Catalog events (ProductCreated, StockUpdated, etc.) remain In-App only. Identity, Order, Logistics, and Payment events are routed to Email channel. Admins can also create manual email notifications.

15. AI Augmentation — Exception Playbooks

The platform includes an AI-powered decision support layer for operational exception handling. This is designed as a **human-in-the-loop** system that recommends actions but requires operator approval for execution.

15.1 AI Use Cases

CAPABILITY	DESCRIPTION	PHASE
Exception Triage	Priority scoring — which issue to resolve first	Phase 1
Next Best Action	Recommendations from approved playbook catalog	Phase 1
Impact Estimation	Delay risk, service impact, cost impact before execution	Phase 2
Natural Language Explanation	Why a recommendation was generated	Phase 1
Post-Resolution Learning	Root cause tagging and recurring pattern detection	Phase 2
Auto-Execution (Low Risk)	Limited auto-execution with post-action review	Phase 3

15.2 AI Architecture

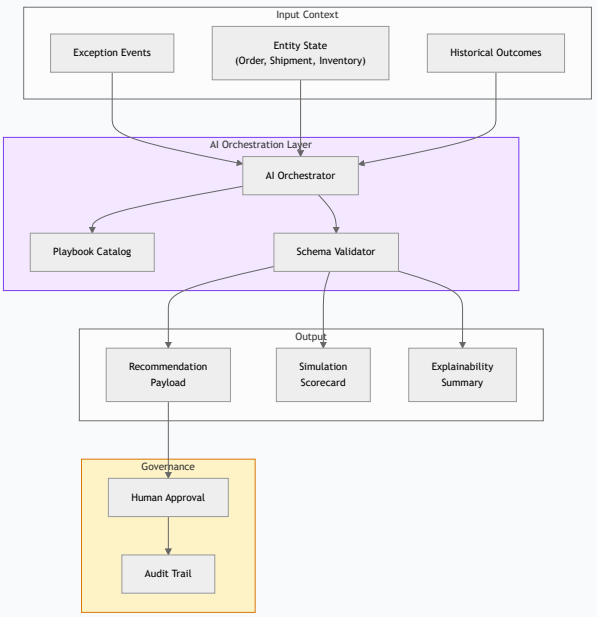

FIGURE 8 — AI EXCEPTION RESOLUTION ARCHITECTURE WITH HUMAN-IN-THE-LOOP GOVERNANCE

15.3 Recommendation Contract

FIELD	TYPE	DESCRIPTION
ExceptionId	GUID	Reference to the operational exception
PlaybookType	string	Category of resolution playbook
SuggestedActions[]	array	Ordered list of recommended actions
ConfidenceScore	float (0.0–1.0)	AI confidence in the recommendation
ExpectedImpact.etaReductionMinutes	int	Estimated time saved
ExpectedImpact.serviceRiskBefore	string	Risk level before action
ExpectedImpact.serviceRiskAfter	string	Risk level after action
ExpectedImpact.costDeltaEstimate	decimal	Cost impact estimate
ExplanationText	string	Natural language reasoning
RequiresHumanApproval	boolean	Whether operator must approve before execution

15.4 Guardrails & Governance

- AI output is validated against strict schema before any action is visible or executable
- Policy rules are applied **after** AI suggestion (role permissions, ownership checks, domain constraints)
- Human approval required for medium and high impact changes
- Auto-fallback to deterministic rule-based playbook if AI fails, times out, or confidence < threshold
- Full audit trail: prompt context hash, model version, recommendation, approver, execution result
- AI provider key stored in environment variable — never in source control

15.5 Phased Rollout

PHASE	MODE	HUMAN INVOLVEMENT
Phase 1	AI Suggest Only	Human approves ALL actions
Phase 2	AI + Simulation Scorecard	Human approves medium/high impact only
Phase 3	Limited Auto-Execution	Low-risk auto-execute with post-action review

16. Risks & Mitigations

RISK	IMPACT	LIKELIHOOD	MITIGATION
RabbitMQ downtime causes event loss	High	Low	Outbox pattern ensures events are persisted before publish; dispatcher retries
Internal API keys exposed	High	Low	Environment variable storage; network-level access control planned
Single-region SQL Server failure	Critical	Low	Regular backups; multi-region considered for future phases
Token theft enables unauthorized access	High	Medium	Short-lived tokens, Redis revocation store, refresh token rotation
Notification email delivery failure	Medium	Medium	Failure-state tracking; retry with dead-letter fallback
AI recommendation quality degrades	Medium	Medium	Confidence threshold + auto-fallback to rule-based playbooks

17. Architectural Decisions Record (ADR)

ID	DECISION	RATIONALE	STATUS
ADR-001	Microservice architecture	Isolate business domains for independent deployment and scalability	Accepted
ADR-002	Outbox + RabbitMQ for events	Guarantee at-least-once delivery with transactional outbox model	Accepted
ADR-003	Ocelot as API Gateway	Lightweight .NET-native gateway with built-in JWT, rate limiting, LB	Accepted
ADR-004	SQL Server per service	Strong transactional integrity; team familiarity with EF Core	Accepted
ADR-005	Redis for idempotency	Sub-millisecond key lookups for duplicate request prevention	Accepted
ADR-006	Angular standalone components	Simplified dependency management; signals for reactive state	Accepted
ADR-007	QuestPDF for invoices	.NET-native PDF generation without external dependencies	Accepted
ADR-008	MediatR pipeline behaviors	Clean separation of cross-cutting concerns from business logic	Accepted
ADR-009	AI as decision-support, not decision-maker	Human-in-the-loop governance for operational safety; phased rollout	Accepted

Supply Chain Management Platform — High Level Design Document v2.0

Confidential · April 2026 · Page generated for print/PDF export